Efficient Transformers and Beyond

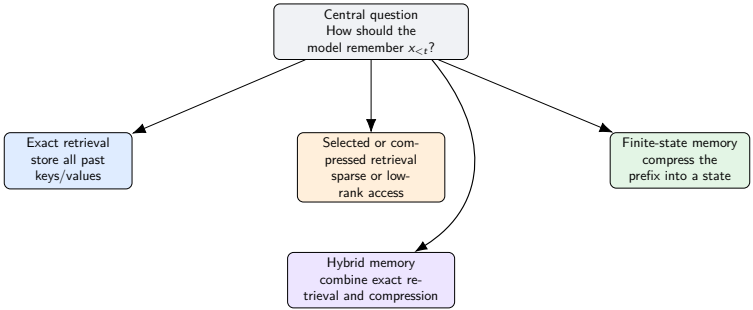
Memory Design for Sequence Models

What this lecture is really about

- A sequence model always has to answer one question: *how should it remember the prefix $x_{<t}$?*
- The standard Transformer answers with exact retrieval from a growing KV cache.
- Efficient Transformers and beyond-Transformer models change either:
  - what is stored about the past,
  - how that stored past is updated,
  - or how it is read at training and inference time.
- By the end, you should be able to classify model families, derive the finite-state view behind modern recurrent methods, and explain why recent practice favors strong exact baselines plus hybrid memory designs.

Bottom line: this is a lecture about *memory design*, not a list of disconnected model names.

Four memory styles organize the field



- **Exact retrieval:** Transformer, FlashAttention.
- **Compressed retrieval:** Longformer, BigBird, Linformer, Nyströmformer, MLA.
- **Finite state:** RNN, LSTM, linear attention, RetNet, Mamba, xLSTM.
- **Hybrid:** Griffin, Jamba, Zamba, Titans, Kimi Linear.

"Efficient" is not one number

Axis	Question	Why it matters
Training sequence cost	How does work grow with sequence length n?	Exact attention has an $n \times n$ interaction pattern; many alternatives aim for subquadratic or linear behavior.
Inference memory	What must be stored while decoding?	A growing KV cache can dominate deployment cost even when training is manageable.
Hardware efficiency	Does the operator map well to GPU/TPU kernels?	Better asymptotics can still lose if kernels have poor memory traffic or low utilization.
Latency / throughput	How fast is long-context decoding in practice?	This is often the systems bottleneck that users actually feel.
Quality	Does the model still retrieve, reason, and extrapolate well?	Compression only helps if task-relevant information survives.

Bottom line: always separate *architecture*, *cache design*, and *kernel implementation*.

Running toy memory task

We will reuse one tiny task throughout the deck.

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad q = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

- The query q should recover something close to v_1 because it aligns much more with k_1 than with k_2.
- We will ask the same two questions for every architecture:
  - **What is stored?** all past items, a compressed summary, or a fixed state?
  - **How is it read?** exact similarity search, approximate retrieval, or state readout?
- This one toy task removes much of the mystery: the formulas change, but the memory problem does not.

The baseline task is causal language modeling

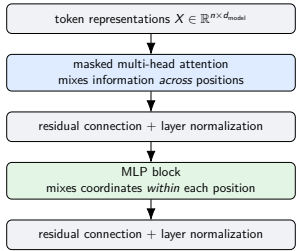
$$\mathcal{L}(\theta) = -\sum_{t=1}^{n} \log p_\theta(x_t \mid x_{<t}).$$

- $x_{1:n}$ is the token sequence; $x_{<t}$ is the prefix before token t.
- **Interpretation:** at each step, the model must turn the prefix into a useful internal memory for next-token prediction.
- **Why this matters:** every architecture in this lecture is a different answer to the same optimization problem:

$$\text{accurate next-token prediction} \quad + \quad \text{efficient prefix representation.}$$

Mental model: the whole field is about how to summarize $x_{<t}$ well enough and cheaply enough.

What a standard decoder block does



- Token embeddings map discrete tokens to vectors in $\mathbb{R}^{d_{\text{model}}}$.
- Residual connections let each block add a correction rather than rebuild the whole representation.
- Layer normalization stabilizes optimization; the MLP is tokenwise, while attention is cross-token.

Causal self-attention from first principles

For one head, learned matrices produce

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

For token t,

$$y_t = \sum_{j \le t} \alpha_{tj} v_j, \qquad \alpha_{tj} = \frac{\exp\left(q_t^\top k_j / \sqrt{d_k}\right)}{\sum_{\ell \le t} \exp\left(q_t^\top k_\ell / \sqrt{d_k}\right)}.$$

- q_t is the **query**: what token t is looking for.
- k_j is the **key**: how past token j should be matched.
- v_j is the **value**: the content returned if past token j is relevant.
- The condition $j \le t$ is the **causal mask**: token t may read the past, not the future.
- **Interpretation:** compare q_t with all permitted keys, normalize the scores into weights, then average the corresponding values.
- **Why this matters:** exact attention is a differentiable retrieval rule over the visible prefix.

A tiny numerical attention head

Take

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad q = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Then

$$q^\top k_1 = 2, \qquad q^\top k_2 = 0,$$

so the weights are approximately

$$\alpha_1 \approx 0.88, \qquad \alpha_2 \approx 0.12.$$

The output is

$$y \approx 0.88 \begin{bmatrix} 3 \\ 1 \end{bmatrix} + 0.12 \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.64 \\ 1.12 \end{bmatrix}.$$

- **Interpretation:** the query mostly retrieves the value attached to the first key.
- **Why this matters:** exact attention is just compare $\rightarrow$ normalize $\rightarrow$ average.

Where exact attention pays its costs

During full-sequence training the score matrix is

$$QK^\top \in \mathbb{R}^{n\times n}, \qquad \text{work} \sim \mathcal{O}(n^2 d_k), \qquad \text{score storage} \sim \mathcal{O}(n^2).$$

Training

- Every query position can interact with every visible key position.
- The bottleneck is the dense pairwise interaction pattern over the sequence.

Autoregressive inference

- At step t, past keys and values must remain available.
- The cache grows roughly like

$$\mathcal{O}\big(t\, N_{KV}(d_k + d_v)\big),$$

where N_{KV} is the number of distinct cached key-value sets.

Keep these separate

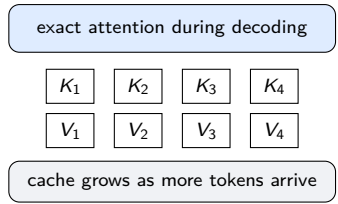
Quadratic *training* cost comes from many token–token interactions. Large *inference* memory comes from storing the past for reuse.

Why positional information is necessary

- Without position, attention only sees a *set* of tokens, not their order.
- Common strategies:
  - **Additive positional embeddings:** add a learned or sinusoidal position vector.
  - **RoPE (Rotary Position Embedding):** rotate query/key coordinates so relative position changes the inner product.
  - **ALiBi (Attention with Linear Biases):** add a distance-dependent score bias.
- **Interpretation:** positional information changes the similarity rule itself.
- **Why this matters:** efficient variants still need ordered sequence information even when they change the memory mechanism.

Why the KV cache becomes a deployment bottleneck

- During decoding, old keys and values are reused instead of recomputed.
- Long contexts therefore make *cache width per token* a systems bottleneck.
- This is why MQA, GQA, and MLA became central in modern deployment work.
- They do not abandon exact retrieval; they reduce how expensively the past is stored.



Bottom line: exact retrieval stays powerful, but the stored past becomes a first-class systems design variable.

Efficient Transformer methods differ by what they change

What changes?	Core move	Representative methods
Attention pattern	Restrict which key positions each query can inspect	local attention, Longformer, BigBird, Reformer
Sequence-direction dimensionality	Compress the key/value sequence into fewer slots	Linformer, Nyströmformer
Kernel	Replace softmax similarity by a feature-space inner product	linear attention, Performer
Implementation	Keep exact attention, but execute it in a better way	FlashAttention-1/2/3
Cached inference state	Keep exact retrieval, but store the past more economically	MQA, GQA, MLA

Bottom line: many papers look different because they operate on different parts of the same pipeline.

Sparse attention: exact retrieval over fewer edges

The simplest sparse rule is a local window:

$$y_t = \sum_{j=\max(1,t-w+1)}^{t} \alpha_{tj} v_j.$$

- w is the window size: token t only compares with the most recent w positions.
- **Interpretation:** keep exact attention on the permitted edges, but make the visibility graph sparse.
- **Why this matters:** when $w \ll n$, the work is about $\mathcal{O}(nw)$ rather than $\mathcal{O}(n^2)$.
- **Main strength:** strong local modeling and straightforward implementation.
- **Main limitation:** a far token is not directly retrievable unless extra edges are added.

Representative sparse families

- **Longformer:** local windows plus a few designated global tokens, so special positions can broadcast document-level information.
- **BigBird:** local, random, and global edges; random links shorten graph distance while global tokens act as hubs.
- **Reformer:** locality-sensitive hashing groups similar tokens into buckets, then mostly attends within each bucket instead of all pairs.

Low-rank attention: compress the sequence dimension

Linformer projects the sequence direction before attention:

$$\tilde{K} = E_K K, \qquad \tilde{V} = E_V V, \qquad E_K, E_V \in \mathbb{R}^{k \times n}, \quad k \ll n.$$

Then attention is computed against the compressed slots:

$$Y \approx \mathrm{softmax}\left(\frac{Q\tilde{K}^\top}{\sqrt{d_k}}\right)\tilde{V}, \qquad Q \in \mathbb{R}^{n \times d_k},\ \tilde{K} \in \mathbb{R}^{k \times d_k},\ \tilde{V} \in \mathbb{R}^{k \times d_v}.$$

- n is original sequence length; k is the learned bottleneck dimension.
- **Interpretation:** do not compare each query with all original positions if the attention map is approximately low-rank.
- **Why this matters:** if the low-rank assumption is valid, training cost can become near-linear in n up to the bottleneck size.
- **Representative families:** Linformer uses learned projections; Nyströmformer uses landmark-based interpolation.
- **Limitation:** highly irregular or sharply localized attention may not compress well.

From softmax attention to a kernel view

For one token t, exact attention can already be written as

$$y_t = \frac{\sum_{j \le t} \kappa_{\mathrm{soft}}(q_t, k_j) v_j}{\sum_{j \le t} \kappa_{\mathrm{soft}}(q_t, k_j)}, \qquad \kappa_{\mathrm{soft}}(q, k) = \exp\left(\frac{q^\top k}{\sqrt{d_k}}\right).$$

- **Interpretation:** softmax attention is a normalized weighted sum, where the weights come from a similarity kernel.
- The kernelized view keeps the same numerator-over-denominator structure, but replaces κ_{soft} by a kernel that can be written or approximated as

$$\kappa(q, k) \approx \phi(q)^\top \phi(k).$$

- **Why this matters:** once the similarity is an inner product in feature space, the prefix sums can be reorganized into running states.

Where ϕ comes from: exact infinite-dimensional view

Important: ϕ is derived for the *unnormalized score kernel*

$$\kappa_{\mathrm{soft}}(q,k) = \exp\left(\frac{q^\top k}{\sqrt{d_k}}\right),$$

not for the whole normalized softmax ratio at once. Use the exponential power series:

$$\exp\left(\frac{q^\top k}{\sqrt{d_k}}\right) = \sum_{m=0}^{\infty} \frac{1}{m!}\left(\frac{q^\top k}{\sqrt{d_k}}\right)^m.$$

Since

$$(q^\top k)^m = \langle q^{\otimes m},\, k^{\otimes m}\rangle.$$

an exact infinite feature map is

$$\phi_\infty(q) = \left[1,\ \frac{q}{d_k^{1/4}},\ \frac{q^{\otimes 2}}{\sqrt{2!}\, d_k^{1/2}},\ \frac{q^{\otimes 3}}{\sqrt{3!}\, d_k^{3/4}},\ \cdots\right],$$

and it satisfies

$$\phi_\infty(q)^\top \phi_\infty(k) = \exp\left(\frac{q^\top k}{\sqrt{d_k}}\right).$$

Takeaway: in theory, ϕ comes from factorizing the exponential kernel via its power-series expansion, but the exact feature map is infinite dimensional.

Where ϕ comes from: practical finite versions

Story 1: approximate the softmax score kernel. Performer uses positive random features:

$$\phi(q) = \frac{1}{\sqrt{m}} \left[e^{\omega_1^\top q - \|q\|^2/2}, \dots, e^{\omega_m^\top q - \|q\|^2/2} \right],$$

where ω_i are sampled random projection vectors, so that

$$\mathbb{E}\left[\phi(q)^\top \phi(k)\right] = e^{q^\top k}.$$

- **Interpretation:** this is a Monte Carlo approximation to the exponential score kernel.
- Smaller m is cheaper but noisier; larger m is more faithful but more expensive.

Story 2: choose ϕ directly. Some linear-attention variants use a positive map such as $\phi(x) = \mathrm{ELU}(x) + 1$.

- Then ϕ is *not* approximating softmax; it defines a different kernel and therefore a different attention rule.

Bottom line: ϕ can mean either a softmax-kernel approximation or a hand-chosen new kernel feature map.

Kernelized linear attention: the bridge to finite-state memory

Assume the similarity can be written or approximated as

$$\kappa(q, k) \approx \phi(q)^\top \phi(k).$$

Then the attention numerator becomes

$$\sum_{j \le t} \phi(q_t)^\top \phi(k_j) v_j = \phi(q_t)^\top \left(\sum_{j \le t} \phi(k_j) v_j^\top \right).$$

- The same pull-out trick will also turn the denominator into a small running state.
- **Interpretation:** the whole prefix can be summarized by one matrix state instead of a growing list of past tokens.
- **Why this matters:** linear attention is simultaneously an efficient-attention idea *and* a recurrent-memory idea.
- **Representative methods:** linear attention, Performer.
- **Limitation:** replacing softmax by a feature map usually weakens exact retrieval behavior.

Preview: this state view becomes the technical center of the lecture.

Linear attention: numerator and denominator as states

Start from the normalized form

$$y_t = \frac{\sum_{j \le t} \kappa(q_t, k_j) v_j}{\sum_{j \le t} \kappa(q_t, k_j)}.$$

With $\kappa(q, k) \approx \phi(q)^\top \phi(k)$, define

$$S_t = \sum_{j \le t} \phi(k_j) v_j^\top, \qquad z_t = \sum_{j \le t} \phi(k_j).$$

$$\sum_{j \le t} \kappa(q_t, k_j) v_j \approx \phi(q_t)^\top S_t, \qquad \sum_{j \le t} \kappa(q_t, k_j) \approx \phi(q_t)^\top z_t.$$

So the output can be written as

$$y_t = \frac{\phi(q_t)^\top S_t}{\phi(q_t)^\top z_t}, \qquad S_t = S_{t-1} + \phi(k_t) v_t^\top, \quad z_t = z_{t-1} + \phi(k_t).$$

- S_t stores the numerator information; z_t stores the normalization information.
- **Why this matters:** the visible prefix is now summarized by two running states rather than a growing KV cache.

FlashAttention: keep exact attention, change the kernel schedule

Naïve attention materializes the full score matrix and the softmax probabilities in slow memory. FlashAttention instead computes exact attention *in tiles*.

- Maintain for each query row:
  - a running maximum m,
  - a running denominator ℓ,
  - and a running weighted-value accumulator o.
- Process blocks of keys and values, update (m, ℓ, o) online, and never store the full attention matrix.
- **Interpretation:** exact softmax attention can be reorganized to reduce high-bandwidth-memory traffic.
- **Why this matters:** better asymptotics are not the only route to speed; a better exact kernel can strengthen the baseline dramatically.

Critical distinction

FlashAttention does *not* approximate attention. It computes the same exact function more efficiently.

MQA, GQA, and MLA target inference memory directly

Names: MQA = multi-query attention, GQA = grouped-query attention, MLA = multi-head latent attention.

At decoding step t, the cache size per layer scales roughly like

$$\text{cache} \propto t \times N_{KV} \times (d_k + d_v),$$

where N_{KV} is the number of distinct cached key-value sets.

- **Interpretation:** exact retrieval gets cheaper when many query heads share or reconstruct the same stored past.
- **Why this matters:** these methods attack the long-context deployment bottleneck even when the attention rule stays exact.

Method	What is cached?	Main benefit	Main trade-off
Full MHA	one key/value set per query head	maximal head flexibility	largest KV cache
MQA	one shared key/value set for all query heads	smallest cache and fast decoding	can be too restrictive
GQA	one key/value set per head group	strong quality–cache compromise	some head specialization is lost
MLA	a latent cache reconstructed into head-specific keys/values	very large cache savings at long context	more architectural complexity

Concrete example

With 32 query heads, full multi-head attention stores 32 KV sets. Grouping heads in fours gives 8 sets instead.

Full MHA vs GQA: where the sharing happens

For one token representation x_t, think of three steps:

$$x_t \xrightarrow{\text{query projection}} q_t^{(h)} \qquad x_t \xrightarrow{\text{KV projection / compression}} \text{cache write} \qquad \text{cache} \xrightarrow{\text{read / reconstruct}} \text{attention}.$$

- **Full MHA:** each head h gets its own $q_t^{(h)}, k_t^{(h)}, v_t^{(h)}$, and the cache stores one (k, v) pair per head.
- **GQA:** queries still stay per-head, but keys/values are produced per group g, so several heads read the same cached $(k_t^{(g)}, v_t^{(g)})$.

$$\text{Full MHA: } k_t^{(h)} = x_t W_K^{(h)},\ v_t^{(h)} = x_t W_V^{(h)}, \qquad \text{GQA: } k_t^{(g)} = x_t W_K^{(g)},\ v_t^{(g)} = x_t W_V^{(g)}.$$

- **Important:** GQA does not first make per-head (k, v) pairs and then merge them; the shared group cache is produced directly by group-level projections, and head h reads group $g(h)$.
- **MQA:** same step changes as GQA, but more aggressively: it is the one-group case, so all heads share one cached (k, v) set.
- **MLA:** cache a latent code $c_t = x_t W_C$, then reconstruct head-specific keys/values on read, e.g.

$$k_t^{(h)} = A_K^{(h)} c_t, \qquad v_t^{(h)} = A_V^{(h)} c_t.$$

Cross-axis synthesis of efficient-Transformer ideas

Family	Main bottleneck targeted	What stays exact?	Typical tradeoff
Sparse attention	training interaction count	retrieval on kept edges	may miss useful long-range edges
Low-rank attention	sequence-direction dimensionality	global attention after compression	depends on low-rank structure
Linear attention	training cost and decode state	feature-space inner-product retrieval	weaker exact softmax behavior
FlashAttention	hardware efficiency	full softmax attention	same model, different kernel
MQA, GQA, MLA	decode-time cache size	exact retrieval from shared or compressed cache state	less per-head flexibility or added reconstruction complexity

Transition: if the KV cache is still the bottleneck, the next question is whether the prefix can be stored in a fixed-size state again.

RNNs had the right memory budget but the wrong bottleneck

A vanilla recurrent neural network (RNN) uses

$$h_t = \sigma(W_h h_{t-1} + W_x x_t).$$

- h_t is a fixed-size hidden state summarizing the entire past.
- **Interpretation:** the memory size does not grow with sequence length.
- **Why this matters:** fixed-state decoding is exactly what modern efficient models want.
- **Main weakness:** the whole prefix is repeatedly squeezed into one vector and long-range information can decay rapidly.

For the scalar linearized recurrence

$$h_t = a h_{t-1} + b x_t,$$

information from k steps ago is scaled by a^k. If $|a| < 1$, it vanishes exponentially.

LSTM: add a memory lane and gates

The long short-term memory (LSTM) update is

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t.$$

- c_t is the cell state, f_t the forget gate, i_t the input gate, and g_t the candidate write.
- **Interpretation:** the old memory can be copied forward additively instead of being remixed from scratch.
- **Why this matters:** gradients can flow through time much more easily when the memory path is additive.

Example: if $c_{t-1} = 4$, $f_t = 0.9$, $i_t = 0.1$, and $g_t = 2$, then

$$c_t = 0.9 \cdot 4 + 0.1 \cdot 2 = 3.8.$$

Most of the old memory survives because the forget gate is near 1.

From bottlenecked compression to explicit retrieval

- **Fast weights:** an early hint that memory can be an associative store built from outer-product-style writes.

Bahdanau attention made the seq2seq move explicit:

$$c_t = \sum_s \alpha_{ts} h_s^{\text{enc}}, \qquad \alpha_{ts} = \frac{\exp\big(\text{score}\big(h_t^{\text{dec}}, h_s^{\text{enc}}\big)\big)}{\sum_r \exp\big(\text{score}\big(h_t^{\text{dec}}, h_r^{\text{enc}}\big)\big)}.$$

- h_s^{enc} is encoder state s; h_t^{dec} is the current decoder state.
- **Interpretation:** the decoder no longer compresses the whole source sentence into one vector; it retrieves the source positions most relevant at time t.
- **Why this matters:** explicit retrieval solved the old seq2seq bottleneck.
- **Transformer:** self-attention generalizes the same idea to every token and makes retrieval highly parallel.

State-space intuition: recurrence can be viewed as a filter

A simple state-space model is

$$h_t = ah_{t-1} + bx_t, \qquad y_t = ch_t.$$

Unrolling gives

$$h_t = bx_t + abx_{t-1} + a^2 bx_{t-2} + \cdots .$$

- **Interpretation:** a recurrence is a causal convolution with exponentially decaying weights.
- **Why this matters:** modern recurrent and state-space models keep reusing this basic idea, but with richer states and smarter update rules.
- A continuous-time model
$$\dot{h}(t) = Ah(t) + Bx(t)$$
becomes a discrete recurrence after discretization.

First-pass takeaway: fixed-state models summarize the past through learned filters rather than explicit token retrieval.

Continuous time to discrete recurrence: one-step derivation

Start from the continuous-time linear state equation

$$\dot{h}(t) = Ah(t) + Bx(t).$$

Over one small step of length Δ, approximate the derivative by a finite difference:

$$\dot{h}(t) \approx \frac{h_t - h_{t-1}}{\Delta}.$$

Plugging this into the differential equation gives

$$\frac{h_t - h_{t-1}}{\Delta} \approx Ah_{t-1} + Bx_t.$$

Multiply by Δ and rearrange:

$$h_t \approx (I + \Delta A)h_{t-1} + (\Delta B)x_t.$$

Same pattern: "old state carried forward + current input written in."
Scalar case: $h_t \approx (1 + \Delta A)h_{t-1} + (\Delta B)x_t$, so it matches $h_t = ah_{t-1} + bx_t$ with $a \approx 1 + \Delta A$ and $b \approx \Delta B$.
Why this matters: $\dot{h}(t) = Ah(t) + Bx(t)$ is the continuous-time view; $h_t = ah_{t-1} + bx_t$ is the sampled recurrence executed token by token.

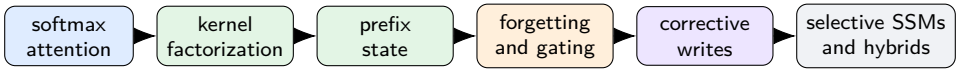
Why the pre-2023 roots mattered

These lines are easiest to remember by *which weakness of old recurrence they tried to fix*.

Line	Core idea	Why it mattered later
HiPPO	keep coefficients of the past in a principled basis	showed that fixed-state compression can be mathematically derived, not only heuristically designed
S4	parameterize structured state matrices so long filters are trainable and practical	made state-space models serious long-sequence backbones rather than only elegant theory
H3	diagnose why early SSMs struggled on language: weak recall and weak comparison	directly motivated later content-sensitive designs
Hyena	combine very long convolutions with multiplicative data-controlled gating	showed that attention-free sequence mixing can still be expressive at long context
RWKV	pair Transformer-like parallel training with recurrent fixed-state decoding	made constant-state language-model inference feel practical again

Bottom line: the 2023–2026 revival inherited principled compression, structured filters, language-specific diagnostics, and scalable recurrent decoding from these earlier lines.

One derivational chain explains many modern models



- Linear attention, RetNet, GLA, DeltaNet, Gated DeltaNet, Mamba, and xLSTM are easiest to learn as *changes to one common memory update story*.
- The common question is not "which paper came first?" but rather "what finite state is kept, and how is it updated?"

Step 1: start from causal attention

The exact readout is

$$y_t = \frac{\sum_{j \le t} \exp(q_t^\top k_j)\, v_j}{\sum_{j \le t} \exp(q_t^\top k_j)}.$$

- The numerator is a similarity-weighted sum of values.
- The denominator normalizes those similarities into weights.
- **Interpretation:** the current query interacts with *every* earlier key.
- **Why this matters:** the full prefix stays explicitly accessible, but the interaction pattern is expensive and the stored past keeps growing.

Goal of the framework: preserve useful prefix information while replacing the growing explicit memory by a finite state.

Step 2: kernelization exposes a prefix state

Suppose the similarity can be approximated by a feature-space inner product:

$$\exp(q^\top k) \approx \phi(q)^\top \phi(k).$$

Then

$$\sum_{j \le t} \phi(q_t)^\top \phi(k_j) v_j = \phi(q_t)^\top \left(\sum_{j \le t} \phi(k_j) v_j^\top \right).$$

Define the prefix state

$$S_t = \sum_{j \le t} \phi(k_j) v_j^\top, \qquad z_t = \sum_{j \le t} \phi(k_j).$$

Readout becomes

$$y_t = \frac{S_t^\top \phi(q_t)}{z_t^\top \phi(q_t)}.$$

- S_t is a matrix-valued memory; z_t keeps the normalizer.
- **Why this matters:** the whole prefix has become one finite state.

Step 3: linear attention is a recurrent state update

From the state definition,

$$S_t = S_{t-1} + \phi(k_t)v_t^\top, \qquad z_t = z_{t-1} + \phi(k_t).$$

- **Interpretation:** each new token adds one outer product to the memory and one feature vector to the normalizer.
- **Why this matters:** linear attention is not only "attention made cheaper"; it is a recurrent memory model.

Using the toy task and taking ϕ as the identity just for intuition,

$$S_2 = k_1 v_1^\top + k_2 v_2^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Then $S_2^\top q = q$, so the readout favors the first association.

RetNet idea: add fixed forgetting

A simple decay gives

$$S_t = \lambda S_{t-1} + k_t v_t^\top, \qquad 0 \le \lambda \le 1.$$

Unrolling yields

$$S_t = \sum_{j=1}^{t} \lambda^{t-j} k_j v_j^\top.$$

- **Notation note:** here k_t can be read as the already-featured key; equivalently, this is the $\phi = \mathrm{id}$ case of the earlier linear-attention write.
- λ is the retention factor: old items are multiplied by λ^{t-j}.
- **Interpretation:** the memory is attention-like, but it has an explicit recency bias.
- **Why this matters:** fixed forgetting gives a clean time-scale control and keeps inference recurrent.
- **Limitation:** one fixed decay schedule may be too rigid for diverse content.

GLA idea: let the model choose what to keep

Replace the fixed decay by a gate:

$$S_t = G_t \odot S_{t-1} + k_t v_t^\top.$$

- G_t is a data-dependent gate, often diagonal or elementwise in practice.
- **Interpretation:** different parts of the memory can decay at different rates depending on the current token.
- **Why this matters:** content-dependent forgetting is more expressive than a single global retention factor.
- RetNet asks *how much should everything fade?*; GLA asks *what specifically should survive?*

Conceptual shift

The jump from fixed decay to gating is small in notation but large in modeling power.

DeltaNet: correct the memory rather than blindly add to it

First predict the value associated with the current key:

$$\hat{v}_t = S_{t-1}^{\top} \phi(k_t).$$

Then write only the residual error:

$$S_t = S_{t-1} + \eta_t \, \phi(k_t) \big(v_t - \hat{v}_t\big)^{\top}.$$

- S_{t-1} is the current matrix memory and η_t is a write step size.
- **Interpretation:** if the memory already predicts well, the update is small; if it is wrong, the correction is large.
- Compare with the additive update

  $$S_t = S_{t-1} + \phi(k_t) v_t^{\top},$$

  which writes the full observation even when the memory already contains it.
- **Why this matters:** corrective writes use limited recurrent memory more efficiently than unconditional outer-product accumulation.

Why $S_{t-1}^{\top}\phi(k_t)$ is a value estimate

Start from the stored memory

$$S_{t-1} = \sum_{j<t} \phi(k_j) v_j^{\top}.$$

Then the DeltaNet prediction is

$$\hat{v}_t = S_{t-1}^{\top}\phi(k_t) = \left(\sum_{j<t} \phi(k_j) v_j^{\top}\right)^{\top} \phi(k_t).$$

Expand and regroup:

$$\hat{v}_t = \left(\sum_{j<t} v_j \phi(k_j)^{\top}\right) \phi(k_t) = \sum_{j<t} v_j \left(\phi(k_j)^{\top}\phi(k_t)\right).$$

- So $\hat{v}_t$ is a similarity-weighted combination of past values.
- If the current key looks like earlier keys, their values contribute strongly.
- **Why this matters:** $S^{\top}\phi(k)$ acts like the memory's best guess of which value should go with the current key.

A generic state-space form unifies many fixed-state models

A broad template is

$$h_t = A_t h_{t-1} + B_t x_t, \qquad y_t = C_t h_t.$$

Unroll the recurrence:

$$\begin{aligned} h_t &= A_t h_{t-1} + B_t x_t \\ &= A_t\big(A_{t-1} h_{t-2} + B_{t-1} x_{t-1}\big) + B_t x_t \\ &= A_t A_{t-1} h_{t-2} + A_t B_{t-1} x_{t-1} + B_t x_t \\ &= \cdots = \sum_{j \le t} \left(\prod_{i=j+1}^{t} A_i \right) B_j x_j. \end{aligned}$$

Then read out:

$$y_t = C_t h_t = \sum_{j \le t} C_t \left(\prod_{i=j+1}^{t} A_i \right) B_j x_j.$$

- A_t carries old state forward, B_t writes the current input, and C_t reads out.
- **Interpretation:** the model is a structured lower-triangular operator on the input history.
- **Why this matters:** RetNet, GLA, DeltaNet, and selective SSMs can all be viewed through this template.

RetNet as a concrete instance of the generic template

RetNet's matrix-memory update is

$$S_t = \lambda S_{t-1} + k_t v_t^\top.$$

To match the generic state-space form, flatten the matrix state:

$$h_t = \mathrm{vec}(S_t).$$

Then the same update can be written as

$$h_t = A_t h_{t-1} + B_t x_t \qquad \text{with} \qquad A_t = \lambda I, \;\; B_t x_t = \mathrm{vec}(k_t v_t^\top).$$

- The state is the matrix memory S_t.
- The fixed forgetting factor λ is exactly the transition A_t.
- The new key–value association $k_t v_t^\top$ is the current write term $B_t x_t$.
- **Why this matters:** RetNet looks special in its own notation, but structurally it is just one simple point in the broader state-space family.

Mamba and Mamba-2: selective SSMs in two views

SSM = state-space model; SSD = structured state-space duality. Here Δ_t is a token-dependent discretization step, so from continuous time $\dot{h}(\tau) = Ah(\tau) + Bx(\tau)$, one token step gives $\bar{A}_t = e^{\Delta_t A}$.
A simplified selective SSM update is

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t, \qquad \bar{A}_t = e^{\Delta_t A}.$$

- h_t is the hidden state; $\bar{A}_t$ retains and $\bar{B}_t$ writes input.
- **Selection** means the current token changes the update itself through Δ_t.
- **Why this matters:** unlike time-invariant SSMs, Mamba makes memory content dependent.

Equivalent causal operator:

$$y_t = \sum_{j \le t} M_{tj} x_j, \qquad M_{tj} = C_t \left(\prod_{i=j+1}^{t} \bar{A}_i \right) \bar{B}_j.$$

Mamba-2 / SSD view: this structured lower-triangular operator supports chunked kernels.

Where one-token discretization comes from

Start from

$$\dot{h}(\tau) = Ah(\tau) + Bx(\tau), \qquad x(\tau) = x_t \text{ on one token interval of length } \Delta_t.$$

Use variation of constants: let $h(\tau) = e^{\tau A} z(\tau)$. Then

$$\dot{h}(\tau) = Ae^{\tau A} z(\tau) + e^{\tau A} \dot{z}(\tau) = Ah(\tau) + Bx_t \;\Rightarrow\; \dot{z}(\tau) = e^{-\tau A} Bx_t.$$

Integrate from 0 to Δ_t:

$$z(\Delta_t) = z(0) + \int_0^{\Delta_t} e^{-sA} Bx_t \, ds, \qquad z(0) = h_{t-1}.$$

Multiply by $e^{\Delta_t A}$:

$$h_t = e^{\Delta_t A} h_{t-1} + e^{\Delta_t A} \int_0^{\Delta_t} e^{-sA} ds \, Bx_t = e^{\Delta_t A} h_{t-1} + \left(\int_0^{\Delta_t} e^{(\Delta_t - s)A} ds \right) Bx_t.$$

So define

$$\bar{A}_t := e^{\Delta_t A}, \qquad \bar{B}_t := \left(\int_0^{\Delta_t} e^{(\Delta_t - s)A} ds \right) B, \qquad h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t.$$

Takeaway: Δ_t controls how far we flow under the same continuous matrix A; in selective SSMs it depends on the current token, so $\bar{A}_t$ and $\bar{B}_t$ become content dependent.

xLSTM: LSTM gating plus matrix-valued associative memory

A matrix-memory xLSTM variant uses

$$
\begin{aligned}
C_t &= f_t C_{t-1} + i_t v_t k_t^\top, \\
n_t &= f_t n_{t-1} + i_t k_t, \\
h_t &= o_t \odot \frac{C_t q_t}{\max(1,\ |n_t^\top q_t|)}.
\end{aligned}
$$

- C_t is the matrix memory; f_t, i_t, o_t are forget, input, and output gates.
- n_t is a normalization state; q_t is the current read query.
- **Interpretation:** xLSTM keeps durable LSTM-style gating, but the cell now stores many key–value associations rather than only one vector.
- **Why this matters:** it sits at the intersection of LSTMs, fast weights, and linear-attention-style matrix memory.

Why recurrent models can still train in parallel

A generic affine recurrence is

$$s_t = A_t s_{t-1} + b_t.$$

Represent one step by the map $\mathcal{T}_t(s) = A_t s + b_t$. Then

$$(A_2, b_2) \circ (A_1, b_1) = (A_2 A_1,\ A_2 b_1 + b_2).$$

- **Interpretation:** segment summaries can be composed associatively.
- **Why this matters:** prefix-scan and chunkwise algorithms can recover training parallelism even for recurrent models.
- This is a major reason modern recurrent architectures are far more practical than old textbook RNN implementations.

Worked example: a 4-token affine scan tree

Take the scalar recurrence

$$s_t = 2s_{t-1} + b_t, \qquad \mathcal{T}_t(s) = 2s + b_t \iff (A_t, b_t) = (2, b_t).$$

Compose neighboring steps in parallel:

$$\mathcal{T}_{2:1} = (2, b_2) \circ (2, b_1) = (4,\ 2b_1 + b_2), \qquad \mathcal{T}_{4:3} = (2, b_4) \circ (2, b_3) = (4,\ 2b_3 + b_4).$$

Then compose those chunk summaries:

$$\mathcal{T}_{4:1} = (4,\ 2b_3 + b_4) \circ (4,\ 2b_1 + b_2) = (16,\ 8b_1 + 4b_2 + 2b_3 + b_4).$$

So the whole 4-token block is summarized by

$$s_4 = 16s_0 + 8b_1 + 4b_2 + 2b_3 + b_4.$$

- **Parallel scan:** compute the summaries for tokens $1{:}2$ and $3{:}4$ at the same time, then merge them.
- **Takeaway:** scan trees recover GPU-friendly parallel training even though the underlying update is causal.

Unified comparison of update laws

Family	Core update	Main extra ingredient	What problem it targets
Linear attention	additive matrix state	feature-map kernelization	replace growing token memory by a finite state
RetNet	additive state + decay	fixed forgetting factor	control time scale with explicit recency bias
GLA	additive state + gate	data-dependent forgetting	keep or erase different memory parts selectively
DeltaNet	corrective update	delta-rule residual write	use limited memory more efficiently
Gated DeltaNet	forgetting + correction	two control mechanisms	combine time-scale control with self-correction
Mamba / Mamba-2	selective SSM recurrence	token-dependent dynamics, SSD view	make SSM memory content aware and hardware friendly
xLSTM	gated matrix memory	modernized LSTM-style gating	preserve durable gating intuition with richer memory

Bottom line: recent model names mostly differ in *how the finite state is updated*, not in the existence of a completely new principle.

Four shifts changed the frontier

1. **Exact attention became a much stronger baseline.**
   - FlashAttention-2 and FlashAttention-3 made exact softmax far more hardware efficient.
2. **Inference cache efficiency moved to the center.**
   - GQA and MLA made long-context decoding more practical.
3. **Finite-state models became credible alternatives.**
   - RetNet, Mamba, DeltaNet, and xLSTM made fixed-state memory competitive again.
4. **Hybrids became the practical compromise.**
   - Exact retrieval and compressed memory solve different problems well, so many systems now combine them.

Recent exact-attention progress strengthened the baseline

Cluster	Representative works	What improved	What stayed the same
Exact kernels	FlashAttention-2, FlashAttention-3	hardware utilization, memory traffic, end-to-end exact-attention speed	the model still performs standard softmax retrieval
Cache sharing	GQA	long-context decode throughput and cache size	still exact attention, still retrieval based
Latent cache compression	MLA in DeepSeek-style systems	cache size at very long context	still not a recurrent state; retrieval information is reconstructed from compressed latents

Frontier lesson: recent progress did not simply move away from Transformers; it also made the exact-attention baseline much harder to beat.

Finite-state alternatives became credible at scale

Mechanism cluster	Representative models	What they promise	Typical concern
Fixed forgetting	RetNet	recurrent inference with explicit retention time scales	fixed decays may be too rigid
Data-dependent forgetting	GLA, HGRN2-style lines	more expressive control of what survives	gating design and parallelization details matter
Selective SSMs	Mamba, Mamba-2	strong linear-time sequence modeling with content-dependent state updates	behavior is less interpretable than plain attention
Corrective writes	DeltaNet, Gated DeltaNet	better use of limited memory by writing residual errors	exact pinpoint retrieval is still hard
Matrix gated memory	xLSTM	durable LSTM intuition with richer state structure	design space is more complex than classic LSTM

Unifying point: these are all attempts to make a fixed-size state expressive enough to compete with a growing exact-retrieval cache.

Hybrids became the practical compromise

Hybrid motif	Representative models	Why it exists	What is mixed
Local attention + recurrence	Griffin, Hawk, RecurrentGemma, Samba	keep precise nearby access while using cheap long-range state	sliding-window or local attention with recurrent blocks
Interleaved Transformer–Mamba	Jamba, Jamba-1.5, Zamba, Zamba2	exact retrieval helps some layers more than others	full attention layers alternating with selective SSM layers
Within-layer / head-level mixing	Hymba, Falcon-H1	reduce cache pressure without abandoning attention entirely	attention and SSM-style components inside the same layer or head set

Why hybrids are so common

Exact attention is excellent at pinpoint retrieval. Recurrent state is excellent for compact long-range memory. Many workloads need both.

Ultra-long context pushed models toward explicit memory

- **Titans:** add a learned long-term memory module alongside attention.
- **MiniMax-01 and related systems:** combine large-scale architectures with operators designed for million-token context regimes.
- **Kimi Linear:** combine linear-delta memory with MLA-style cache compression.

These systems point to a further question:

Should long history live in a separate explicit memory bank?

- **Interpretation:** sometimes neither a pure KV cache nor a pure finite state is enough.
- **Why this matters:** very long context becomes a systems problem as much as an architecture problem.

What the frontier did *not* decide

- It did **not** find one universal replacement for the Transformer.
- It did **not** show that asymptotically linear models always win in practice.
- It did **not** make exact retrieval obsolete.

What it **did** show:

- exact attention remained stronger than many expected once kernels improved,
- fixed-state models became much more credible once gating, correction, and selection matured,
- hybrid designs attracted practical mindshare because different memory types solve different bottlenecks.

Master comparison across memory families

Family	What is stored?	Training sequence cost	Inference state	Main strength / main limitation
Exact attention	all past keys and values	typically quadratic in n	growing KV cache	strongest direct retrieval / expensive at long context
Sparse attention	selected retrieval edges	usually sub-quadratic	structured cache	cheap local access / may miss long dependencies
Low-rank attention	compressed sequence summaries	often near-linear in used bottleneck	compressed summaries	useful when attention is low-rank / weak on irregular patterns
Linear attention and recurrent memory	finite matrix or vector state	linear in n	fixed-size state	cheap long decoding / weaker exact retrieval
Selective SSMs	finite state with input-dependent dynamics	linear in n	fixed-size state	strong efficiency–quality tradeoff / harder to interpret
Hybrids	some exact retrieval plus some compressed memory	depends on the mix	mixed cache + state	often best compromise / more design complexity
Explicit memory systems	base model plus separate memory module	depends on base system	state plus external memory	very long-history handling / extra systems complexity

Five design rules that recur across the literature

1. **Ask first whether the task needs exact retrieval.**
   - Pinpoint copying from arbitrary positions is still hard for pure fixed-state models.
2. **Separate training complexity from inference memory.**
   - A model can train efficiently yet still carry an expensive decode-time state, or the reverse.
3. **Read decays, gates, and step sizes as time-scale controls.**
   - They decide what old information survives.
4. **Corrective and selective updates try to make limited memory more expressive.**
   - DeltaNet and Mamba are two different examples of this principle.
5. **Hybrids appear when one memory type is not enough.**
   - This is the practical reason they keep returning.

A practical decision guide

If your setting mainly needs...	Prefer to start from...	Reason
arbitrary pinpoint retrieval over long prefixes	exact attention or cache-efficient exact attention	the task demands exact addressability of many past positions
mostly local context with occasional global hubs	sparse attention	most useful edges are nearby, so explicit sparsity is natural
long-context decoding with strict memory budget	finite-state recurrent models or selective SSMs	inference state stays fixed instead of growing with context
a strong quality–efficiency compromise	hybrids	exact retrieval and compressed memory can be used where each helps most
very long history beyond a single window or state	explicit memory modules	a separate memory bank can store what neither the KV cache nor a fixed state can hold gracefully

First question to ask about any new paper: what is stored, and how is it read?

Final takeaway

- **Exact retrieval:** strongest direct access to the past, but expensive in training interactions and decode-time cache.
- **Finite-state memory:** fixed inference state, but it must work harder to preserve precise long-range information.
- **Modern practice:** stronger exact-attention systems, smarter recurrent models, and hybrids that combine both.

Best guiding question:
What should be stored about the past for this task and this hardware?

Appendix: minimum notation

Symbol	Meaning	Typical shape or role
X	input token representations	$n \times d_{\text{model}}$ matrix of token vectors
Q, K, V	queries, keys, values	usually $n \times d_k$, $n \times d_k$, $n \times d_v$
q_t, k_t, v_t	one token's query, key, value	vectors for token t
S_t	matrix-valued recurrent memory	often feature-dimension by value-dimension
z_t	linear-attention normalizer state	feature-sum vector used for normalization
h_t	vector-valued recurrent state	hidden state of an SSM or recurrent model
KV cache	stored past keys and values	grows with context length in exact attention

Appendix: the key FlashAttention derivation

For one query row with scores s_j, keep a running maximum m, denominator ℓ, and numerator accumulator o.

$$m^{\text{new}} = \max\left(m^{\text{old}}, m^{\text{blk}}\right).$$

$$\ell^{\text{new}} = e^{m^{\text{old}} - m^{\text{new}}} \ell^{\text{old}} + \sum_{\text{blk}} e^{s_j - m^{\text{new}}}.$$

$$o^{\text{new}} = e^{m^{\text{old}} - m^{\text{new}}} o^{\text{old}} + \sum_{\text{blk}} e^{s_j - m^{\text{new}}} v_j.$$

Finally,

$$y = \frac{o^{\text{final}}}{\ell^{\text{final}}}.$$

- **Interpretation:** exact softmax can be accumulated block by block without storing the full attention matrix.
- **Why this matters:** the derivation proves FlashAttention is exact, not approximate.

Appendix: why sparse patterns with hubs can still communicate globally

BigBird-style sparsity combines

- local window edges,
- a few random edges,
- and a few global tokens.

Path-length intuition:

- if each token can reach a global token in one hop,
- and the global token can reach every other token in one more hop,
- then long-range communication can happen in very few layers even without full dense attention.

Why this matters: sparse attention is not only about fewer edges; it is about designing a useful communication graph.

Appendix: two low-cost approximation ideas

Nyströmformer

$$A(Q,K) \approx A(Q,\tilde{K})\, A(\tilde{Q},\tilde{K})^{\dagger}\, A(\tilde{Q},K).$$

- Landmark queries/keys $(\tilde{Q},\tilde{K})$ approximate the full kernel matrix.

Performer / positive random features

$$\mathbb{E}\big[\phi(q)^{\top}\phi(k)\big] = e^{q^{\top}k}.$$

- The softmax kernel is approximated by low-variance random features, which then plug into linear-attention recurrences.

Appendix: DeltaNet from one-step regression

Use the one-step loss

$$\mathcal{L}_t(S) = \tfrac{1}{2}\|S^\top \phi(k_t) - v_t\|_2^2.$$

Its gradient is

$$\nabla_S \mathcal{L}_t(S) = \phi(k_t)\big(S^\top \phi(k_t) - v_t\big)^\top.$$

A gradient step gives

$$S_t = S_{t-1} + \eta_t \phi(k_t)\big(v_t - r_t\big)^\top, \qquad r_t = S_{t-1}^\top \phi(k_t).$$

A useful rearrangement is

$$S_t = \big(I - \eta_t \phi(k_t)\phi(k_t)^\top\big) S_{t-1} + \eta_t \phi(k_t) v_t^\top.$$

- **Interpretation:** correct the memory in the current key direction, then write the improved association.

Appendix: selective state-space discretization

A simplified Mamba discretization writes

$$\bar{A}_t = e^{\Delta_t A}, \qquad \bar{B}_t = \left(\int_0^{\Delta_t} e^{(\Delta_t - \tau)A} \, d\tau \right) B(x_t).$$

- Δ_t is an input-dependent step size.
- **Interpretation:** the token changes the effective transition and write rule.
- **Mamba-2 / SSD view:** the resulting causal operator can be represented as a structured lower-triangular semiseparable matrix, which supports chunked hardware-friendly algorithms.

Appendix: self-check questions

Try answering these without looking back.

1. Why is FlashAttention not an "attention replacement"?
2. In one sentence, what is the difference between GQA and a recurrent state?
3. What assumption makes low-rank attention plausible?
4. What does the normalizer state z_t do in linear attention?
5. Why does a delta-rule write waste less memory than a pure additive write?
6. What does "selective" mean in selective state-space models?
7. Why do hybrids keep appearing even after strong recurrent models were developed?

Appendix: compact glossary

Term	Meaning in this deck
Exact retrieval	Keep past keys/values explicitly and read them by attention similarity.
KV cache	Stored past keys and values used during autoregressive decoding.
Linear attention	Kernelized attention whose prefix can be summarized by a finite recurrent state.
Retention	Linear-attention-style matrix memory with explicit decay.
Selective SSM	State-space model whose transition or write rule depends on the current token.
Corrective write	Update that writes a prediction residual rather than a raw value.
Hybrid memory	Architecture that mixes exact retrieval with recurrent state or explicit long-term memory.

Appendix: classical SSM discretization in one place

A continuous-time linear SSM is

$$\dot{h}(t) = Ah(t) + Bu(t), \qquad y(t) = Ch(t) + Du(t).$$

Under zero-order hold over a step of length Δ,

$$h_t = \bar{A}h_{t-1} + \bar{B}u_t, \qquad \bar{A} = e^{\Delta A}, \qquad \bar{B} = \int_0^{\Delta} e^{(\Delta-\tau)A} B \, d\tau.$$

- A controls continuous dynamics; B writes the input; C reads out.
- **Interpretation:** discretization turns a differential equation into the recurrence actually used by sequence models.
- Unrolling gives

$$y_t = \sum_{j=1}^{t} C\bar{A}^{\,t-j}\bar{B}\, u_j + Du_t,$$

so linear SSMs are also causal convolutions.

Appendix: how frontier papers usually support their claims

Claim type	Typical evidence in the source survey
Faster exact attention	kernel benchmarks, model-FLOPs utilization, end-to-end training or inference speedups
Cheaper cache without quality loss	cache-size analysis, long-context throughput, uptraining or matched-quality comparisons
Better finite-state memory	language-model perplexity, synthetic memory tasks, long-context evaluations, ablations on gates and corrections
Better hybrids	compute-matched benchmark suites, mixture ablations, extrapolation to longer contexts
Ultra-long-context claims	needle-in-a-haystack tests, million-token throughput, cross-domain long-history tasks

Why this matters: frontier papers make different kinds of claims, so they should not all be read with the same evaluation lens.